



FTF – Survey Invoicing & Billing (E2E)

AI Invoicing Agent — **Internal** Documentation Package

Client: NexGen Surveying / FieldToFinish

Prepared by: Tisko Tech

Version: 1.0 · **Date:** 2026-07-16

Status: Internal — Draft for review

CONFIDENTIAL — INTERNAL USE ONLY · NOT FOR CLIENT DISTRIBUTION

Project Charter, RAID Log & Decision Register

Field	Value
Project	FTF – Survey Invoicing & Billing Delivery (E2E) · Client NexGen Surveying / FieldToFinish
Document	Project Charter, RAID Log & Decision Register · Version 1.0 · Status Internal
Prepared by	Tisko Tech · Owner Prateek Chandra · Updated 2026-07-16
Audience	Delivery team, PMO, engineering leads

Internal document. Contains delivery detail not shared with the client. No secrets/keys.

Table of Contents

- 1. Purpose & Scope
- 2. Project Charter
- 3. Governance & Team
- 4. RAID Log (Risks / Assumptions / Issues / Dependencies)
- 5. Decision Register
- 6. Change Request Register
- 7. Approval & Revision History

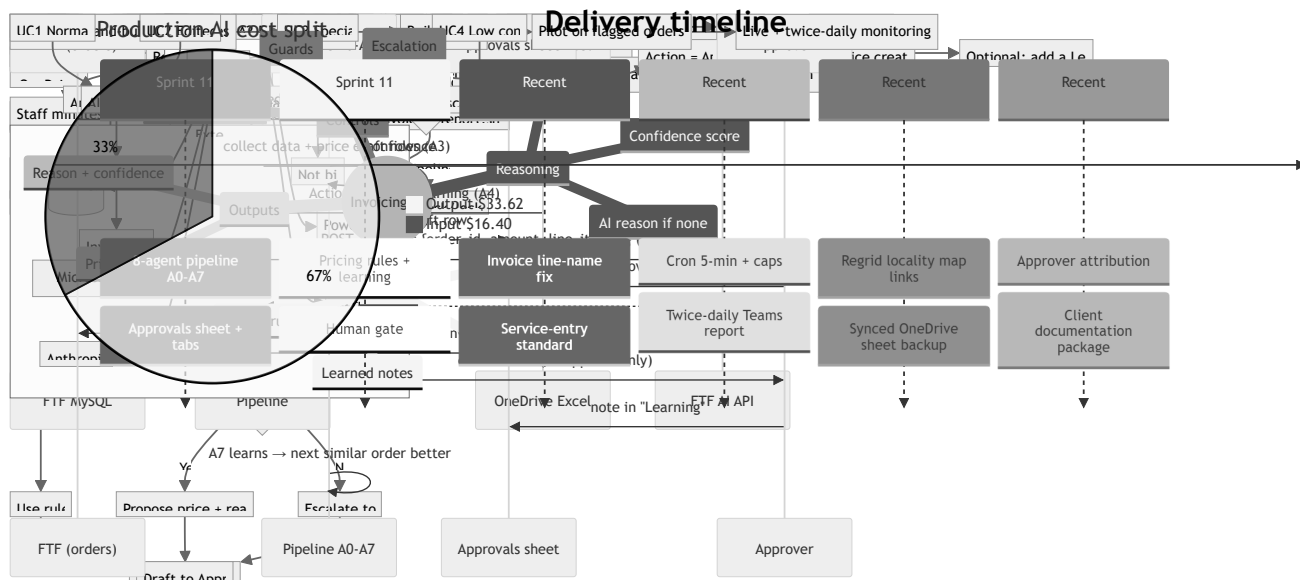
1. Purpose & Scope

Give the delivery team one control document: why the project exists, who owns what, what could go wrong (RAID), what we decided, and what changed.

In scope: the automated pipeline that finds FTF orders needing an invoice, drafts a priced invoice, routes it to a human for approval, creates the real invoice in FTF, emails the client, and learns from edits. **Out of scope:** quotes, marketing, crew scheduling (future phases).

2. Project Charter

Item	Detail
Problem	Invoices were built by hand — slow, inconsistent pricing, orders occasionally missed for billing.
Goal	AI drafts every flagged invoice; a human only checks & approves.
Success measures	~0 flagged orders missed · consistent rule-based pricing · seconds to draft · run cost < \$1/order.
Solution type	AI Agent pipeline (8 agents A0–A7) + OneDrive approval sheet + Teams reporting.
Sponsor	[CLIENT INPUT REQUIRED] — NexGen sponsor name.
Delivery owner	Tisko Tech (Prateek Chandra).



3. Governance & Team

Role	Owner	Responsibility
Delivery / Tech lead	Prateek Chandra	Architecture, deploy, sign-off
Client operations	Sumit Parik + NexGen staff	Approve/reject on the sheet
Domain (surveying)	FL PLS advisor	Pricing rules, service types
AI provider	Anthropic (Claude)	Model runtime

Cadence: async status via twice-daily Teams report; escalations ad-hoc.

4. RAID Log

Risks

ID	Risk	Impact	Likelihood	Mitigation
R1	AI mis-prices an unusual job	Wrong bill	Med	Human approval gate + escalate on low confidence
R2	Excel Online caches an open file → edits not seen	Stale run	Med	Self-heal next run; guide users to reopen
R3	FTF never flags an order <code>invoice_needed</code>	Order not surfaced	Low	Documented; option to broaden intake
R4	Duplicate invoice created	Double bill	Low	Pre-create re-check against FTF
R5	Anthropic/Graph/SMTP outage	Pipeline stalls	Low	Read-only report independent; flock guards; retries

Assumptions

- FTF `ng_invoice_needed=1` is the authoritative "bill this" signal.
- Every invoice passes a human approval before creation/sending.
- One production Anthropic key isolates this project's cost.

Issues (resolved)

ID	Issue	Resolution
I1	Edited service name blank on invoice	FTF renders <code>description</code> ; A5 now falls back name→desc→"Survey Service"
I2	Action dropdown missing 3 values	Rebuilt validation on col O (O2:O10000) via openpyxl
I3	Extra "AI Agent Created Invoice" log line	Confirmed server-side (FTF), not ours — left to system

Dependencies

Dependency	Type	Notes
FTF stage API + MySQL	External	Order data + invoice creation
Microsoft Graph (Workbook + activities)	External	Sheet read/write, real editor identity
Anthropic Claude	External	Sonnet 4.6 (run), Opus 4.8 (build)
Power Automate flow (<code>TEAMS_FLOW_URL</code>)	External	Teams delivery
EC2 host + cron	Internal infra	Scheduling

5. Decision Register (delivery decisions)

#	Decision	Rationale	Date
D1	Human approval gate is mandatory; AI never auto-approves/sends	Safety & trust	Sprint 11
D2	Cron cadence 5 min (30 → 15 → 5)	Faster capture of new flagged orders	2026-07
D3	Two Teams monitoring reports/day (12 PM & 7 PM ET)	Operational visibility + by-whom audit	2026-07
D4	Attribute approvals to real editor via Graph activities feed	Accurate "by whom" without new log infra	2026-07
D5	Do NOT write our own FTF order-log lines	"Let the system do it by itself"	2026-07
D6	Service-entry standard <code>Name: \$Amount ...</code>	Edits flow to invoice deterministically	2026-07

6. Change Request Register

CR	Request	Status	Result
CR1	Show edited service name on invoice	 Done	A5 label fallback
CR2	Learn from "Learning provided by user" each run + apply to similar orders	 Done	A3 rule injection strengthened
CR3	Bring 10 named test orders to sheet	 Done	10/10 seeded
CR4	Cron → 5 min	 Done	caps 100/30/30
CR5	Twice-daily Teams report + tabs updated	 Done	deployed + verified

CR	Request	Status	Result
CR6	Approver attribution (Sumit vs Prateek)	 Done	activities feed
CR7	Col-F map link opens the parcel, not search	 Done	Regrid locality URL (new orders)
CR8	Server backup of the full sheet + shareable URL	 Done	local + OneDrive copy, org link, every cycle

Approval

Name	Role	Decision	Date
	Delivery lead	☐ Approve	

Revision history — 1.0 (2026-07-16, Tisko Tech): initial internal charter/RAID/decisions.

Internal Requirements, Business Rules & User Stories

Field	Value
Project	FTF – Survey Invoicing & Billing Delivery (E2E) · Client NexGen Surveying / FTF
Document	Internal Requirements, Business Rules & User Stories · Version 1.0 · Status Internal
Prepared by	Tisko Tech · Owner Prateek Chandra · Updated 2026-07-16
Audience	BA, developers, QA

Table of Contents

1. Purpose & Scope
2. Functional requirements
3. Business rules (pricing & guards)
4. User stories + acceptance criteria
5. Core use cases
6. Workflow (end-to-end)
7. Approval & Revision History

1. Purpose & Scope

The single source for **what the system must do** and **the rules it must obey**. Scope = invoice preparation for FTF orders flagged `invoice_needed`, through human approval, creation, send, and learning.

2. Functional requirements

ID	Requirement	Priority
FR1	Detect FTF orders with <code>ng_invoice_needed=1</code> not yet invoiced	Must
FR2	Collect order data (address, lot size, county, flood zone, service type)	Must
FR3	Propose a price from Pricing Rules → history → AI reasoning, with confidence	Must
FR4	Write a draft row to the OneDrive Approvals tab	Must
FR5	Read the human Action (Approve / On-hold / Reject) each run	Must
FR6	On Approve, create the real invoice in FTF and email the client	Must
FR7	Never create/send without a human Approve	Must
FR8	Read Learning provided by user each run; apply to this + similar orders	Must
FR9	Honor the service-entry standard <code>Name: \$Amount ...</code> so edits reach the invoice	Must
FR10	Guard against duplicate / condo / canceled / delivered orders	Must
FR11	Record the real approver per decision	Must
FR12	Post a monitoring report twice daily (12 PM & 7 PM ET)	Must

ID	Requirement	Priority
FR13	Escalate low-confidence / unusual jobs instead of guessing	Must

3. Business rules

Pricing

- **BR-P1:** A matching Pricing Rule wins over AI reasoning.
- **BR-P2:** No rule → AI reasons from order data + market history + learned notes.
- **BR-P3:** Line total = sum of `Name: $Amount` entries in the **"Service / Breakdown by User"** column (col K); the total (col L) recalculates each run.
- **BR-P4:** Low confidence or unusual job → **Escalate** (manager review), never auto-priced.

Guards (never bill)

Rule	Condition	Action
BR-G1 Duplicate	Order already invoiced in FTF	Skip; do not re-bill
BR-G2 Condo	Order is a condo (out of scope)	Flag "cannot survey"; not billed
BR-G3 Canceled	Order canceled	Flag; not billed
BR-G4 Delivered	Already delivered/closed	Flag; not billed

Learning

- **BR-L1:** A note in "Learning provided by user" is read **every run**.
- **BR-L2:** A note applies to that order **and** similar orders (same client, same/related service, or neighbouring county).
- **BR-L3:** Learning is **additive** — it guides pricing; it never bypasses the approval gate.

Safety

- **BR-S1:** Only a human sets Action; the AI/agent never approves.
- **BR-S2:** No client email is sent except as a result of a human Approve.
- **BR-S3:** The system owns FTF order-log lines; the agent does not write them.

4. User stories + acceptance criteria

#	As a...	I want...	So that...	Acceptance
US1	Ops approver	to see a drafted price per order	I only check, not build	Draft row with price + confidence appears within ~5 min
US2	Ops approver	to edit the service line	the invoice reflects my change	Edited <code>Name: \$Amount</code> prints on the invoice; total updates
US3	Ops approver	to teach the AI in plain words	future bills are better	Note read next run; similar orders repriced
US4	Manager	nothing sent without approval	we stay in control	No invoice/email created for blank/On-hold/Reject

#	As a...	I want...	So that...	Acceptance
US5	Delivery lead	to see who approved what	we have an audit	Twice-daily report lists approver by name

5. Core use cases

6. Workflow (end-to-end)

Approval

Name	Role	Decision	Date
	BA / Product	☐ Approve	

Revision history — 1.0 (2026-07-16, Tisko Tech): initial.

UX, Interface Design & User Flows

Field	Value
Project	FTF – Survey Invoicing & Billing Delivery (E2E) · Client NexGen Surveying / FTF
Document	UX, Interface Design & User Flows · Version 1.0 · Status Internal
Prepared by	Tisko Tech · Owner Prateek Chandra · Updated 2026-07-16
Audience	UI/UX, developers, ops

Table of Contents

- 1. Purpose & Scope
- 2. Interfaces at a glance
- 3. The Approvals sheet (the primary UI)
- 4. Teams monitoring report
- 5. Client invoice email
- 6. User flow (approver)
- 7. Design guidelines
- 8. Approval & Revision History

1. Purpose & Scope

The system has **no custom app UI**. The user works in tools they already know: an **Excel sheet** (approvals), a **Teams chat** (monitoring), and the **client email** (output). This doc defines each surface and the approver's flow.

2. Interfaces at a glance

Surface	Tool	Who	Purpose
Approvals tab	Excel Online	Approver	Review price, edit, set Action
Guide / How-To / Pricing Rules tabs	Excel Online	Approver	Help + rules
Teams report	Teams chat "AI - Invoicing Agent"	Team	Monitor activity + by-whom
Invoice email	Email	Client	Receives the bill

3. The Approvals sheet (primary UI)

Blue columns = human-editable; gray = AI-managed.

Col	Field	Owner
A–F	Order id, client, property, service type, size, map link	AI
G	FEMA Zone (reference)	AI
I / J	Service / Breakdown by AI + Amount by AI (locked baseline)	AI
K	Service / Breakdown by User (Name: \$Amount ...) — invoiced source of truth	Human
L	Amount (\$) by User (auto-calculated from col K)	AI
O	Action (Approve / On-hold / Reject) — dropdown	Human
T	Learning provided by user — free-text teaching note	Human
approved_by, invoice_id, timestamps	audit	AI

Blue (editable) columns are **K (Service / Breakdown by User)**, **O (Action)**, Notes, and **T (Learning)**. Everything else is gray/AI-managed. **Col G is FEMA Zone, not the price cell** — edit the breakdown in **col K**.

Key interactions

- **Action dropdown** (col O, validation O2:O10000) — 3 values Approve / On-hold / Reject.
- **Service-entry standard** — Boundary Survey: \$500.00 | Elevation Certificate: \$150.00 . The typed **name is what prints** on the invoice; changing the number reprices.
- **Learning cell** — plain-English note; read every run.

UX rule: if the dropdown/total looks wrong, the fix is **close + reopen** the workbook (Excel Online cached the open file).

4. Teams monitoring report (12 PM & 7 PM ET)

- **Deterministic block is authoritative** — exact counts/by-whom are computed, not written by the model.
- "By whom" = real editor from the Graph activities feed for human decisions; agents A1–A7 for automated steps.

5. Client invoice email

- Sent **only** after a human Approve.
- Line label = the service **name** the approver typed (never blank).
- Standard FTF invoice template; amount = sum of the col-K "Service / Breakdown by User" lines.

6. User flow (approver)

7. Design guidelines

- **Reuse familiar tools** — no training on a new app.
- **Blue = yours, gray = ours** color contract on the sheet.
- **One action per row** — the Action dropdown is the only commit control.
- **Plain language** in Guide/How-To tabs (Grade-5).
- **No destructive controls** exposed to the user; the AI never has an "approve" control.

Approval

Name	Role	Decision	Date
	UI/UX lead	☐ Approve	

Revision history — 1.0 (2026-07-16, Tisko Tech): initial.

Solution & Technical Architecture

Field	Value
Project	FTF – Survey Invoicing & Billing Delivery (E2E) · Client NexGen Surveying / FTF
Document	Solution & Technical Architecture · Version 1.0 · Status Internal
Prepared by	Tisko Tech · Owner Prateek Chandra · Updated 2026-07-16
Audience	Architects, developers, DevOps, security

Internal. Implementation detail included. No API keys, passwords, or connection secrets.

Table of Contents

- 1. Purpose & Scope
- 2. System context
- 3. Component architecture (A0–A7)
- 4. Data flow
- 5. Data model (FTF + state store)
- 6. Integrations & APIs
- 7. Infrastructure & deployment
- 8. Security architecture
- 9. Environment configuration
- 10. Approval & Revision History

1. Purpose & Scope

The end-to-end technical picture: components, data, integrations, hosting, and security for the invoicing pipeline.

2. System context

3. Component architecture (A0–A7)

Agent	Responsibility	Key detail
A0	Orchestrate the run, caps	run caps 100/30/30
A1	Find <code>ng_invoice_needed=1</code> , not invoiced	intake watermark on <code>ng_id</code>
A2	Collect order data	address, lot, county, flood zone, service
A3	Compile + price	rules → history → AI; injects learned rules for similar orders
A4	Human gate	reads Action + <code>INPUT_APPROVER</code> ; sets <code>approved_by</code>
A5	Finalize	builds FTF line items; label = name→desc→"Survey Service"
A6	Send	creates FTF invoice, emails client (on approve only)
A7	Learn	records edits/notes for next similar order

4. Data flow

5. Data model

FTF (stage MySQL, pymysql DictCursor):

- `ng_order` — orders; `ng_invoice_needed` , `ng_status` , `ng_id` (watermark), client fields.
- `ng_invoice` / invoice records created via API.

State store (OneDrive Excel + local `pipeline_state`):

- **Approvals** tab — `APPROVAL_HEADERS` (20 cols); `_COL_*` derived via `.index()` ; `_COL_ACTION` =14 (col O).
- `pipeline_state` — per-order: `order_id`, `status`, `client_name`, `service_type`, `estimate_amount`, `invoice_id`, `approved_by`, `data_collected_at`, `draft_posted_at`, `invoice_created_at`, `sent_at`, `updated_at` (`get_all_orders()` / `get_orders_by_status()`).

Status funnel: `invoice_needed` → `data_collected` → `pricing_needed` → `invoice_draft_posted` → `(on_hold)` → `invoice_approved` → `invoice_finalized` → `invoice_sending` → `invoice_sent` ; plus `invoice_rejected`, `condo_rejected`, `delivered_flagged`, `canceled_flagged`, `details_missing` .

6. Integrations & APIs

Integration	Endpoint / mechanism	Notes
FTF AI API	<code>POST /invoices {order_id, amount, line_items:[{name,description,amount}]}</code>	Renders <code>description</code> as the line label; server auto-logs the "AI Agent Created Invoice — N Line Items" line
FTF MySQL	<code>core.ftf_mysql._connect()</code> (pymysql DictCursor)	Read orders (<code>ng_order</code> , <code>ng_invoice_needed</code> , <code>ng_id</code> watermark). (<i>The seed helper <code>seed_specific_orders.py</code> normalizes a 6-digit id to "1000"+id ; live intake uses the id verbatim.</i>)
Graph Workbook	read/write Approvals tab	Data validation NOT supported by Graph (v1.0 & beta 400) → use openpyxl
Graph activities	<code>/drives/{drive}/items/{item}/activities</code>	Real human editors → <code>get_recent_editors()</code> , <code>get_current_approver()</code>
Teams	Power Automate HTTP flow (<code>TEAMS_FLOW_URL</code>)	Report delivery; not Graph, not webhook
Anthropic	Claude Sonnet 4.6 (run)	Pricing/reasoning

7. Infrastructure & deployment

Single-runner model (since 2026-06-26). The pipeline runs on the **prod server** — the only host that reaches the private FTF RDS for A1 intake. The equivalent GitHub Actions workflows (`invoice_pipeline.yml` , `excel_approval_watcher.yml` , `approval_poller.yml`) were **disabled at cutover** so there is exactly one runner and one state file (no duplicate invoices/emails). The crontab calls `/home/ubuntu/ftf-invoicing-*.sh` , which are symlinks/wrappers to the `scripts/run_server_*.sh` files.

Item	Value
Host	Prod server <code>FTF-NEAdmin-HA-01</code> (private infra)
Deploy dir	<code>/home/ubuntu/FTF Invoicing Agent</code>
Runtime	<code>.venv/bin/python</code> (server venv, Python 3.10)
Cron (live)	<code>ftf-invoicing-run.sh */5 ; ftf-invoicing-watch.sh every 5 min; ftf-invoicing-report.sh 0 16,17,23,0 UTC (ET-hour guard {12,19})</code>
Wrappers →	<code>scripts/run_server_pipeline.sh</code> , <code>run_server_watcher.sh</code> , <code>run_server_daily_report.sh</code>
Run caps (live <code>.env</code>)	A1 <code>MAX_NEW_PER_RUN=100</code> · A2 <code>MAX_PER_RUN=30</code> · A3 <code>INVOICE_BATCH_SIZE=30</code> (the "100/30/30" per tick)

Item	Value
Concurrency	<code>flock</code> guards prevent overlap

8. Security architecture

- **Secrets** live only in `.env` on the host — never in code, logs, or docs.
- **Least privilege:** the pipeline uses scoped app credentials for Graph + FTF.
- **Human gate** is the primary control: no external side effect (invoice/email) without a human Approve.
- **Approver identity** recorded per decision from the Graph activities feed.
- **Auditability:** every status transition + approver captured in the state store.
- **No PII in the repo:** documents carry no client PII or secrets.

9. Environment configuration

Config via `.env` (keys referenced by name only): FTF API base + DB creds, Graph app creds + drive/item ids, Anthropic key, `TEAMS_FLOW_URL` , SMTP creds, run caps. `INPUT_APPROVER` is set per cycle from `get_current_approver()` .

py3.10 pitfall: f-strings containing backslashes fail — build strings with plain variables (recurring issue in SSH heredocs).

Approval

Name	Role	Decision	Date
	Enterprise architect	☐ Approve	

Revision history — 1.0 (2026-07-16, Tisko Tech): initial.

AI Engineering Guide

Field	Value
Project	FTF – Survey Invoicing & Billing Delivery (E2E) · Client NexGen Surveying / FTF
Document	AI Engineering Guide (Brain · Prompts · Guardrails · Model · Learning) · Version 1.0 · Status Internal
Prepared by	Tisko Tech · Owner Prateek Chandra · Updated 2026-07-16
Audience	AI engineering, developers, QA

Internal. Describes prompt *strategy* and guardrails. Full prompt text is not reproduced here (kept in code). No secrets.

Table of Contents

1. Purpose & Scope
 2. AI Brain (what the AI does)
 3. Model selection & comparison
 4. Reasoning pipeline
 5. Prompt engineering strategy + version history
 6. Tool calling strategy
 7. Memory & learning architecture
 8. Guardrails & hallucination prevention
 9. Confidence, escalation & fallback
 10. Evaluation & monitoring
 11. Token & latency optimization
 12. AI testing strategy
 13. Approval & Revision History
-

1. Purpose & Scope

How the AI reasons about pricing, how it is kept safe and accurate, and how it learns. Scope = pricing/reasoning inside A3, learning in A7, and the guardrails across the pipeline.

2. AI Brain

Purpose: turn an order into a fair, explained price so a human only checks it.

3. Model selection & comparison

Use	Model	Rate /MTok (in/out)	Why
Production (run)	Claude Sonnet 4.6	\$3 / \$15	Fast, cheap, strong enough for structured pricing; no caching needed
Development (build)	Claude Opus 4.8	\$15 / \$75 (cache-write \$18.75, cache-read \$1.50)	Deep reasoning for engineering the system

Sonnet in production keeps per-order cost < \$1. Opus was used only to build.

4. Reasoning pipeline

5. Prompt engineering strategy + version history

- **Grounded prompts:** the model is given only real order data + rules + learned notes; it is instructed to price and explain, not to invent orders/clients.
- **Structured output:** line items with `name` , `description` , `amount` ; a short reason; a confidence label.
- **Learned-rule injection (A3):** learned notes are injected with an explicit instruction to *"apply the SAME logic to any SIMILAR order (same client, same/related service, or neighbouring county)."*

Prompt version	Change
v(prior)	Base pricing prompt (rules + reasoning + confidence)
v(current)	Strengthened learned-rule injection to generalize to similar orders

6. Tool calling strategy

The pipeline is **deterministic code around the model** — the model prices; code does the side effects (DB read, sheet write, invoice create, email). This keeps every external action auditable and gated. The model does **not** directly call FTF or send email.

7. Memory & learning architecture

- **Sources:** the "Learning provided by user" column (read **every run**) and observed edits.
- **Scope of a lesson:** the specific order **and** similar orders (BR-L2).
- **Additive only:** learning never bypasses approval.

8. Guardrails & hallucination prevention

Guardrail	Mechanism
No invented orders/clients	Model works only from real FTF rows
No auto-send	Human approval gate (A4) before any create/email
Duplicate	Re-check FTF before creating
Condo / canceled / delivered	Detected → flagged, never billed
Deterministic totals	Col-G Name: \$Amount summed in code, not by the model
Unusual job	Escalate instead of guess

9. Confidence, escalation & fallback

Confidence	Meaning	Action
● High	Clear rule/history	Quick check, approve
● Medium	Reasonable, verify	Review price
● Low / Escalate	Unusual / thin data	Manager reviews

Fallback: if the model can't price confidently, the order is escalated (not auto-priced) and still surfaces on the sheet for a human.

10. Evaluation & monitoring

- **Monitoring:** twice-daily Teams report shows pipeline snapshot + activity + by-whom.
- **Evaluation signal:** approve-without-edit rate, edit deltas, escalation rate, missed-flag rate (target ~0).
- **Health checks:** `pipeline_health_check.py` (stuck sets), `export_pipeline_json.py` (status funnel).

11. Token & latency optimization

- Single lean model (Sonnet 4.6) in production; **no cache** needed at this volume.
- Prompts carry only the fields needed to price → low input tokens.
- Measured production cost ≈ **\$50.02 total** (~\$26/mo, ~\$0.70/order).

12. AI testing strategy

- Golden orders with known correct prices (rule + non-rule).
 - Guard tests: condo/canceled/duplicate must never bill.
 - Learning test: add a note → verify next similar order reprices.
 - Approval-gate test: blank/On-hold/Reject must never create or email.
-

Approval

Name	Role	Decision	Date
	AI engineering lead	☐ Approve	

Revision history — 1.0 (2026-07-16, Tisko Tech): initial.

Development & Release Tracker

Field	Value
Project	FTF – Survey Invoicing & Billing Delivery (E2E) · Client NexGen Surveying / FTF
Document	Development & Release Tracker (Sprint · Modules · Build · Releases) · Version 1.0 · Status Internal
Prepared by	Tisko Tech · Owner Prateek Chandra · Updated 2026-07-16
Audience	Dev team, PMO

Table of Contents

1. Purpose & Scope
 2. Sprint plan
 3. Module / feature tracker
 4. Build tracker
 5. Release notes
 6. Git reconciliation (honest counts)
 7. Approval & Revision History
-

1. Purpose & Scope

Track what was built, module state, and releases. Numbers are reconciled against git; where a tracker under-records git, this doc says so.

2. Sprint plan

3. Module / feature tracker

Module	State	Notes
A0 Orchestrator	✓ Done	run caps 100/30/30
A1 Flag hunter	✓ Done	ng_invoice_needed + watermark
A2 Data collector	✓ Done	address/lot/county/flood/service
A3 Invoice compiler + pricing	✓ Done	learned-rule injection strengthened

Module	State	Notes
A4 Human gate	✓ Done	reads Action + real approver
A5 Finalizer	✓ Done	line-label fallback fix
A6 Sender	✓ Done	FTF invoice + client email
A7 Feedback learner	✓ Done	reads Learning col each run
Approvals sheet + Guide/How-To/Pricing tabs	✓ Done	v15 / v8
Action dropdown (col O)	✓ Done	O2:O10000
Twice-daily Teams report	✓ Done	12 & 19 ET, activity + by-whom
Approver attribution	✓ Done	Graph activities feed
Client documentation package	✓ Done	13 docs + PDF
Internal documentation package	✓ In progress	this set

4. Build tracker

Feature	Files touched	Result
Invoice line name	<code>agent_a5_invoice_finalizer.py</code>	name→desc→"Survey Service"
Learn similar orders	<code>agent_a3_invoice_compiler.py</code>	generalize to similar orders
Sheet tabs + standard + cadence	<code>onedrive_excel_client.py</code>	Guide v16 / How-To v9; "5 min"; monitoring
Col-F Regrid locality map link	<code>agent_a3_invoice_compiler.py</code>	<code>/us/{state}/{county}/{city}?query=...</code> (new orders only)
Synced sheet backup + URL	<code>backup_sheet.py</code> , <code>onedrive_excel_client.py</code>	local rolling copies + OneDrive backup file w/ org share link
Get-all helper	<code>excel_db.py</code>	<code>get_all_orders()</code>
Twice-daily report	<code>daily_report.py</code> , <code>run_server_daily_report.sh</code>	activity block + labels + guard
Real approver	<code>run_excel_watcher.py</code> , <code>agent_a4_human_gate_v2.py</code>	<code>INPUT_APPROVER</code> → <code>approved_by</code>
Seed test orders	<code>scripts/seed_specific_orders.py</code>	10 orders

5. Release notes (recent commits to `main`)

Commit	Summary
9886958d	Fix invoice line name + strengthen operator learning + document col-G standard
fa6e005f	Add <code>seed_specific_orders.py</code> — bring an explicit set of order IDs to the sheet
de906d34	Twice-daily Teams monitoring report (12 PM & 7 PM ET) with activity/audit
9cb3afe0	Fix remaining stale "30 min" cadence text in Guide/How-To tabs

Commit	Summary
1d98952e	Attribute approvals to the real human editor (Sumit/Prateek), not hard-coded name

6. Git reconciliation (honest counts)

- Repo history is large (~**6,621 commits**), **0 version tags** — releases are tracked by **feature commits**, not semantic tags. Any release number here is derived from commit subjects, not tags. This is stated so no one over-reads a "version count".
-

Approval

Name	Role	Decision	Date
	Dev manager	☐ Approve	

Revision history — 1.0 (2026-07-16, Tisko Tech): initial.

Test Plan, Cases & Results

Field	Value
Project	FTF – Survey Invoicing & Billing Delivery (E2E) · Client NexGen Surveying / FTF
Document	Test Plan, Cases & Results (incl. Bugs, Regression, Security/Perf) · Version 1.0 · Status Internal
Prepared by	Tisko Tech · Owner Prateek Chandra · Updated 2026-07-16
Audience	QA, developers

Table of Contents

- 1. Purpose & Scope
- 2. Test strategy
- 3. Test cases
- 4. Regression results
- 5. Bug tracker
- 6. Security testing
- 7. Performance testing
- 8. QA checklist
- 9. Approval & Revision History

1. Purpose & Scope

Verify the pipeline prices correctly, never bills without approval, honors guards, and learns — plus security/perf basics.

2. Test strategy

3. Test cases

TC	Scenario	Expected	Result
TC1	Order with matching Pricing Rule	Rule price used	✓ Pass
TC2	Order with no rule	AI proposes price + confidence	✓ Pass
TC3	Approver edits col K "Service / Breakdown by User" (Name: \$Amount)	Edited name + total on invoice	✓ Pass

TC	Scenario	Expected	Result
TC4	Blank / On-hold / Reject Action	No invoice, no email	✓ Pass
TC5	Condo order	Flagged, not billed	✓ Pass
TC6	Duplicate order	Not re-billed	✓ Pass
TC7	Learning note added	Next similar order reprices	✓ Pass
TC8	Approve	Invoice created + client emailed	✓ Pass
TC9	Approver identity	Real editor recorded (Sumit/Prateek)	✓ Pass
TC10	Twice-daily report	Correct counts + by-whom, 12 & 19 ET	✓ Pass (dry-run verified)
TC11	Seed 10 named orders	10/10 appear on sheet	✓ Pass
TC12	Action dropdown	3 values present O2:O10000	✓ Pass

4. Regression results

Before each deploy: run the full TC set + `py_compile` changed files on the server. Latest cycle — **all pass**; report verified by dry-run (`--dry-run --label ... --window-hours ...`).

5. Bug tracker

Bug	Root cause	Fix	State
Edited service name blank on invoice	FTF renders <code>description</code> , not <code>name</code>	A5 label fallback <code>name→desc→"Survey Service"</code>	✓ Fixed
Action dropdown missing 3 values	Validation drifted to O16:O10013; file open (423 lock)	Rebuilt O2:O10000 via <code>openpyxl</code> ; <code>close+reopen</code>	✓ Fixed
Graph can't set data validation	API 400 (v1.0 & beta)	Use <code>openpyxl</code> (only path)	✓ Worked around
Stale "30 min" text in tabs	Cadence changed to 15	Replaced all 5 spots	✓ Fixed
Extra FTF order-log line	Server-side FTF log, not ours	Left to system (by design)	✓ Confirmed not a bug
"TEST" \$400 on 1000286750	Manual invoice by Pchandra	Our invoice 362449 clean \$575	✓ Explained
Usage API 400 (limit>31)	1d bucket cap	Split window into ≤ 31 -day pages	✓ Fixed
f-string backslash <code>SyntaxError</code> (py3.10)	Backslash in f-string expr	Plain variables in heredocs	✓ Fixed
"Missed non-invoiced orders" (Sumit)	Flag-driven design, not a miss	AI only surfaces <code>invoice_needed</code> ; 37/37 tracked	✓ Not a bug

6. Security testing

- Confirmed **no secrets** in code/logs/docs; `.env` only on host.

- Approval gate cannot be bypassed by the AI (no auto-approve path).
- Scoped app credentials for Graph + FTF; no over-broad tokens.

7. Performance testing

- Run caps 100/30/30 keep a cycle bounded; cron every 5 min.
- Verified a cycle queued 25 new orders without overrun; `flock` prevents overlap.
- Report is read-only and independent of the pipeline cron — no contention.

8. QA checklist

- ☒ All TCs pass · ☒ py_compile clean · ☒ No secret leakage · ☒ Guards hold · ☒ Approval gate holds · ☒ Report counts match state · ☒ Dropdown present.
-

Approval

Name	Role	Decision	Date
	QA lead	☐ Approve	

Revision history — 1.0 (2026-07-16, Tisko Tech): initial.

Operations Runbook

Field	Value
Project	FTF – Survey Invoicing & Billing Delivery (E2E) · Client NexGen Surveying / FTF
Document	Operations Runbook (Deploy · Monitor · Incident · Backup · DR · SOPs) · Version 1.0 · Status Internal
Prepared by	Tisko Tech · Owner Prateek Chandra · Updated 2026-07-16
Audience	DevOps, on-call, ops

Internal. Host paths and schedules included. No secrets/keys.

Table of Contents

1. Purpose & Scope
2. Deployment runbook
3. Scheduling (cron)
4. Monitoring guide
5. Incident response
6. Backup & recovery
7. Disaster recovery
8. SOPs
9. Approval & Revision History

1. Purpose & Scope

How to deploy, watch, and recover the pipeline. Host = EC2; deploy dir `/home/ubuntu/FTF Invoicing Agent` ; runtime `.venv/bin/python` (3.10).

2. Deployment runbook

Steps

1. Commit to `main` , push (PAT).
2. `scp` changed files into `/home/ubuntu/FTF Invoicing Agent` (preserve paths).

- Cron calls the `/home/ubuntu/ftf-invoicing-*.sh` wrappers (symlinks to `scripts/run_server_pipeline.sh`, `run_server_watcher.sh`, `run_server_daily_report.sh`).
3. `.venv/bin/python -m py_compile <changed>.py`.
 4. Update crontab only if schedule changed; back up crontab to `/tmp` first.
 5. Verify (below).

3. Scheduling (cron) — live

Job (crontab entry)	Schedule	Purpose
<code>ftf-invoicing-run.sh</code> → <code>run_server_pipeline.sh</code>	<code>* /5 * * * *</code>	Pipeline run (A0→A3 intake + backstop). Caps: A1=100 / A2=30 / A3 batch=30. Shares the flock lock with the watcher (both <code>* /5</code>).
<code>ftf-invoicing-watch.sh</code> → <code>run_server_watcher.sh</code>	every 5 min	Approval watcher (A4→A5→A6) / self-heal
<code>ftf-invoicing-report.sh</code> → <code>run_server_daily_report.sh</code>	<code>0 16,17,23,0 * * * UTC</code>	Teams report; ET-hour guard fires only at 12 & 19 ET

All guarded by `flock` (no overlap). Wrappers set `TZ=America/New_York`; the report wrapper exits unless the ET hour ∈ {12,19}.

4. Monitoring guide

- **Primary:** twice-daily Teams report (snapshot + activity + by-whom).
- **On demand:** `pipeline_health_check.py` (stuck sets), `export_pipeline_json.py` (funnel).
- **Green signals:** new orders queued each cycle; approvals flowing; ~0 missed flags.
- **Watch for:** file-lock (423) on the sheet, Graph/Anthropic/SMTP errors in logs.

5. Incident response

- **Safety:** the human gate means a stall never causes a wrong bill — worst case is delay.
- Escalate to delivery lead if unresolved in one cycle.

6. Backup & recovery

- **Source of truth for orders/invoices = FTF** (external, backed up by FTF).
- **State store** (Excel + `pipeline_state`) is rebuildable from FTF flags; a lost run re-detects flagged orders next cycle.
- **Sheet backup** (`scripts/backup_sheet.py`, **every cycle**): downloads the full OneDrive workbook (all tabs) to `<deploy>/backups/` (`latest.xlsx` + rolling snapshots, newest `SHEET_BACKUP_KEEP` =48 kept) **and** mirrors it to

a separate OneDrive file `FTF-Invoicing Agent BACKUP.xlsx` with an **organization share link** (written to `backups/BACKUP_URL.txt`). Read-only w.r.t. the live sheet — never edits/removes an order.

- Back up crontab before edits (`crontab -l > /tmp/cron.bak`).

7. Disaster recovery

- Host loss: redeploy code to a fresh host, restore `.env` , re-add crontab; pipeline re-detects flagged orders. **RPO** $\approx$ one cycle (5 min); **RTO** = redeploy time.
- No irreplaceable local data — FTF holds the authoritative records.

8. SOPs

- **Change cadence:** edit the crontab entry for `ftf-invoicing-run.sh` , back up first (`crontab -l > /tmp/cron.bak`), verify with `crontab -l` .
- **Add a pricing rule:** append a row in the Pricing Rules tab (no deploy).
- **Teach the AI:** note in "Learning provided by user".
- **Never:** write FTF order-log lines by hand; approve/send on the client's behalf.

Approval

Name	Role	Decision	Date
	DevOps / Ops	☐ Approve	

Revision history — 1.0 (2026-07-16, Tisko Tech): initial.

Finance, Cost & Profitability Dashboard

Field	Value
Project	FTF – Survey Invoicing & Billing Delivery (E2E) · Client NexGen Surveying / FTF
Document	Finance, Cost & Profitability Dashboard · Version 1.0 · Status Internal
Prepared by	Tisko Tech · Owner Prateek Chandra · Updated 2026-07-16
Audience	Finance, delivery lead, executives
Measurement window	2026-05-20 → 2026-07-17 (~1.9 months)

Internal. Contains cost/margin detail NOT shared with the client. All figures **measured** (Admin API / transcripts / git) — none estimated. No secrets.

Table of Contents

- 1. Purpose & Scope
- 2. Executive summary (the numbers)
- 3. AI cost dashboard (production)
- 4. Development (build) cost
- 5. Infrastructure, licensing & vendors
- 6. Profitability & pricing guidance
- 7. ROI & break-even
- 8. Provenance & confidence
- 9. Approval & Revision History

1. Purpose & Scope

The internal money view: what the AI costs to run, what it cost to build, other costs, and margin/pricing guidance. Client-facing ROI lives in the client `AI_Cost_Pricing_and_ROI` doc — this internal version adds margin and pricing that the client does not see.

2. Executive summary

Metric	Value	Confidence
Production run cost (to date)	\$50.02 (~\$26/mo)	High (key-isolated)
Cost per order (run)	~ \$0.72	Illustrative (÷ ~69 tracked)
Development (build) value	\$385.74 (local sessions)	Measured; partial scope
Infra / licensing	see \$5	[CLIENT INPUT REQUIRED] for exact host bill

3. AI cost dashboard (production) — MEASURED

Source: Anthropic **Admin Usage API**, filtered to the `FTF-Invoicing Agent` key. Model **Claude Sonnet 4.6**; no cached tokens.

Item	Tokens	Rate /MTok	Cost
Input	5,467,559	\$3	\$16.40
Output	2,241,239	\$15	\$33.62
Total			\$50.02

- Monthly run rate $\approx$ **\$26/mo**; annualized $\approx$ **\$312/yr** at current volume.
- Per order $\approx$ **\$0.72** ($\$50 \div \sim 69$ orders; illustrative).

4. Development (build) cost — MEASURED (local sessions)

Source: this project's Claude Code transcripts, deduped by `message.id`, model **Opus 4.8**.

Item	Tokens	Rate /MTok	Cost
Input	13,300	\$15	\$0.20
Output	629,125	\$75	\$47.18
Cache write	8,680,345	\$18.75	\$162.76
Cache read	117,063,738	\$1.50	\$175.60
Total			\$385.74

⚠️ Only the **locally-retained** transcript is captured (one session file). Exact for those sessions; **true multi-month build cost is higher** and not fully measurable here. This is a list-rate **value**, not an invoice.

5. Infrastructure, licensing & vendors

Item	Vendor	Cost basis	Notes
Compute (EC2 host)	AWS	monthly instance	<code>[CLIENT INPUT REQUIRED]</code> — exact instance/bill
AI model	Anthropic	usage (above)	Sonnet 4.6 prod
Microsoft 365 / Graph	Microsoft	existing client license	OneDrive Excel + Teams

Item	Vendor	Cost basis	Notes
Power Automate flow	Microsoft	existing plan	Teams delivery
Email (SMTP)	client mail	existing	Client invoicing email

Most platforms reuse the client's **existing** M365 licensing → low incremental cost. The only clearly new recurring cost is **AI** (~\$26/mo) + the host instance.

6. Profitability & pricing guidance (internal only)

Lever	Note
Run cost/order	~\$0.72 — negligible vs invoice value
Suggested client pricing	flat monthly managed fee + optional per-order — sized to value, not cost
Margin driver	run cost is a rounding error; margin ≈ managed fee – (AI + host)
Enterprise/scale	cost scales ~linearly with orders; Sonnet keeps unit cost flat

[CLIENT INPUT REQUIRED] — target managed-service fee and support SLA to finalize margin.

7. ROI & break-even

[CLIENT INPUT REQUIRED] — staff minutes per manual invoice and invoices/month convert time-saved into a dollar ROI + payback period.

8. Provenance & confidence

- **Production:** /usage_report/messages filtered by api_key_id , priced at list rates — **high confidence**.
- **Development:** local ~/.claude/projects/<slug>/*.jsonl , deduped by message.id — **measured, partial scope**.
- **Releases:** git shows ~6,621 commits / **0 tags** — releases tracked by feature commits.
- No figure estimated; scope strictly this project.

Approval

Name	Role	Decision	Date
	Finance / Delivery lead	☐ Approve	

Revision history — 1.0 (2026-07-16, Tisko Tech): initial with measured production + dev cost.

Lessons Learned & Internal Closeout

Field	Value
Project	FTF – Survey Invoicing & Billing Delivery (E2E) · Client NexGen Surveying / FTF
Document	Lessons Learned & Internal Closeout · Version 1.0 · Status Internal
Prepared by	Tisko Tech · Owner Prateek Chandra · Updated 2026-07-16
Audience	Delivery team, PMO

Table of Contents

1. Purpose & Scope
2. What was delivered
3. Lessons learned
4. What went well / what to improve
5. Technical debt & follow-ups
6. Closeout checklist
7. Approval & Revision History

1. Purpose & Scope

Capture the internal retrospective and confirm delivery closeout for this phase.

2. What was delivered

3. Lessons learned

What we learned	Action taken
FTF invoice renders <code>description</code> , not <code>name</code>	A5 label fallback name→desc→"Survey Service"
Excel Online caches an open file (423 lock)	Self-heal next run; guide close+reopen
Graph cannot set data validation	Use openpyxl for the Action dropdown
Approver identity isn't per-cell in Graph	Attribute via activities feed
The AI is only as complete as FTF's flag	Documented; option to broaden intake

What we learned	Action taken
py3.10 f-string backslash breaks in heredocs	Build strings with plain variables
Cost must be isolated by API key	Usage API filtered by <code>api_key_id</code> is authoritative
"Missed orders" was flag-driven, not a bug	Explained to client; 37/37 tracked
Let the system own its own logs	Removed our extra FTF order-log line

4. What went well / what to improve

✅ Went well	🔧 Improve
Human gate kept every side effect safe	Durable per-user audit log (deferred)
Deterministic report block (no hallucinated counts)	Broaden intake beyond <code>invoice_needed</code> (optional)
Measured, honest cost reporting	True editable <code>.docx</code> output (needs pandoc)
Learning generalizes to similar orders	Automated golden-order eval harness

5. Technical debt & follow-ups

- Per-user audit log beyond the on-record approver — **deferred** (activities feed used instead).
- True editable Word `.docx` — needs a one-time `pandoc` install (PDF delivered via Edge).
- ROI dollarization pending `[CLIENT INPUT REQUIRED]` staff-time inputs.
- Optional: `03_Project_Assets` folder (branding, sample data, prompt library) not yet populated.

6. Closeout checklist

Item	Done?
Pipeline delivered + deployed (5-min cron)	☒
Client documentation package	☒
Internal documentation package	☒
Measured cost recorded	☒
UAT sign-off (client)	☐ <code>[CLIENT INPUT REQUIRED]</code>
Support model agreed	☐ <code>[CLIENT INPUT REQUIRED]</code>

Sign-off

Name	Role	Decision	Date
	Delivery lead	☐ Accept	

Revision history — 1.0 (2026-07-16, Tisko Tech): initial internal closeout.